

Structural Invariants, Attack-Graph Mathematics, and an Autonomous Red-Team Platform

VALEN Project

Working draft v0.5 — not peer reviewed

Abstract

We present VALEN — *Verification And Active Logic Engine, Neuro-symbolic* — an end-to-end system that spans two disciplines and joins them with one typed graph intermediate representation. On the *defensive* side it is a multi-language static analyzer whose thesis is *measured*: structural invariants of program graphs (spectral, persistent and directed homology, discrete curvature) carry no information beyond taint for injection prioritization (OWASP Benchmark 1.2, 2740 cases, taint MRR 0.894 vs. the fused field’s 0.872, $p = 5 \times 10^{-5}$), but are the *only* signal for authorization (BOLA/IDOR: recall 1.0 / precision 0.90 on crAPI), destructive state cycles (GLMY directed homology: 0.97 vs. 0.065 recall for symmetrized), and trust topology. On the *offensive* side it is an autonomous red-team platform: a multi-user team server that coordinates reconnaissance, Active Directory attack-graph analysis, command and control, credential recovery, phishing and exfiltration across the kill chain, under an explicit authorization and human-in-the-loop gating model. The unifying contribution is a layered *attack-graph mathematics* — Menger min-cut chokepoints, min-cost flow, absorbing Markov-chain hitting probabilities solved exactly by $(I - Q)^{-1}R$, Z3 Optimize bounded-model-checking that *synthesizes* minimal-cost ordered exploit plans with AND-preconditions, and a probabilistic context-free grammar for password guessing — each answering a question that reachability and shortest-path queries cannot. We report the system deployed against OWASP crAPI (18/18 challenges solved autonomously, including three LLM prompt-injection challenges) and a synthetic Active Directory corpus, with all components installable, tested (383 tests), and documented.

Contents

1	Introduction	1
2	System architecture	2
3	Structural verification	3
3.1	The boundary on geometric priors	3
3.2	Authorization, mechanized and operational	3
3.3	Directed path homology	3
3.4	The neuro-symbolic arbitration boundary	4
4	Multi-language SAST	4
5	The red-team platform	5

6	Attack-graph mathematics	5
6.1	Flow: chokepoints and disjoint plans	6
6.2	Probability: hitting time	6
6.3	Formal synthesis: minimal-cost exploit plans	6
6.4	Password guessing: PCFG	7
7	Evaluation	7
7.1	Autonomous pentest against OWASP crAPI	7
7.2	Active Directory toy corpus	7
7.3	Results at a glance	7
8	Limitations and ethics	8
9	Conclusion	8

1 Introduction

Two questions organize this work. *First, where do structural invariants of program graphs actually carry security information, and where do they not? Second, can the same mathematical machinery be turned into a quantitative planning engine for offensive operations?* The answer to the first is a sharp, statistically-settled boundary (Section 3); the answer to the second is an operational platform whose differentiator is mathematics, not scripting (Sections 5, 6).

Our contributions are:

- C1. A measured boundary on geometric priors for injection SAST** (Section 3.1): with a paired permutation test and bootstrap confidence intervals we show spectral/topological/geometric invariants are at or below chance for injection prioritization, redirecting effort to where structure *is* the signal.
- C2. An operational authorization invariant** (Section 3.2): a privilege-level function mechanized in Lean 4, a structural BOLA/IDOR detector, and a Z3 ownership witness that separates object-level authorization from mere authentication — recovering every documented BOLA/BFLA endpoint of crAPI.
- C3. Directed path homology as the correct cycle signal** (Section 3.3): GLMY H_1 dominates symmetrized homology for destructive-cycle detection on state machines and smart contracts.
- C4. A measured neuro-symbolic arbitration boundary** (Section 3.4): the LLM proposes, formal methods dispose; detection and ranking are LLM-independent while interpretation changes.
- C5. A multi-language SAST front-end** (Section 4): one generic taint engine drives nine language grammars with per-language profiles.
- C6. An autonomous red-team platform** (Section 5): a multi-user team server with RBAC and live collaboration, spanning the kill chain with an authorization-first gating model.
- C7. Attack-graph mathematics for planning** (Section 6): chokepoints, least-cost and disjoint plans, exact hitting probabilities, minimal-cost exploit synthesis, and a PCFG password prior.

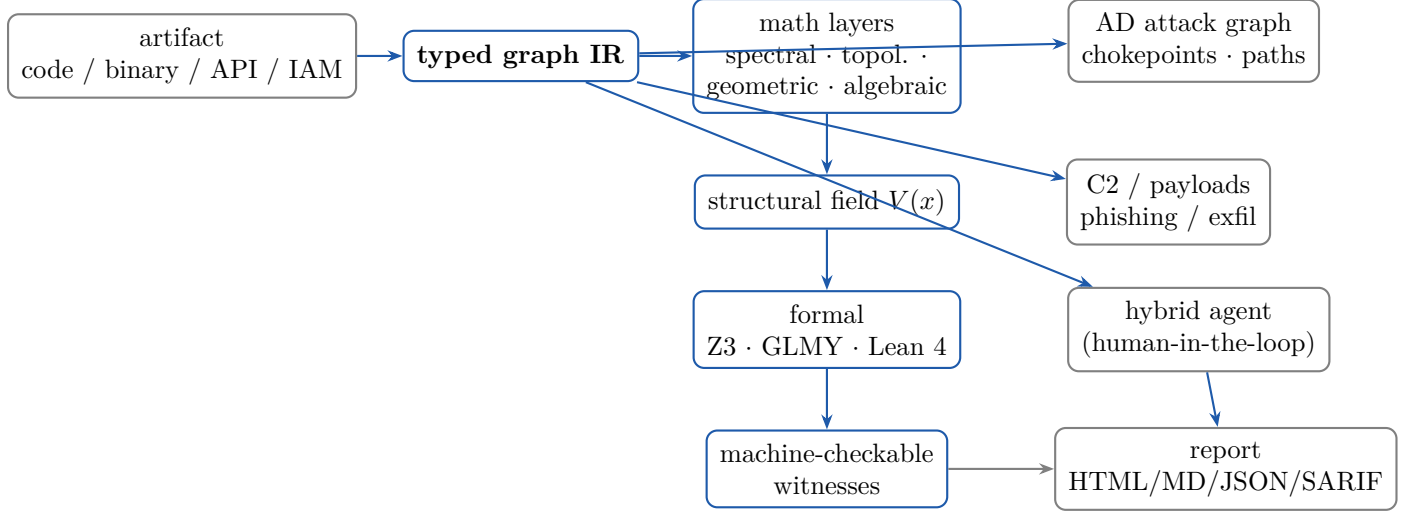


Figure 1: One typed IR feeds both a *verifier* (left) and a *red-team platform* (right). Findings become witnesses, attack plans, and reports.

2 System architecture

Everything is built over one deterministic, JSON-serializable typed graph intermediate representation (IR). Six ingest adapters (Python, Java with interprocedural/context-sensitive taint, C/C++/Rust/C#/Go/PHP/Ruby/JavaScript via a generic C-like engine, binary via `objdump/angr`, OpenAPI, IAM, LLM-agent, Solidity) emit the same graph, shared verbatim between the Python layer and an optional Rust numeric core (with a pure-Python fallback). Five signal families are fused over it — algebraic (taint lattice, privilege levels), spectral (Fiedler), topological (persistent and GLMY directed homology), geometric (Forman–/Ollivier–Ricci), and formal (Z3, Lean 4) — and every result carries a machine-checkable witness under the modeled semantics.

3 Structural verification

3.1 The boundary on geometric priors

On OWASP Benchmark 1.2 (2740 cases with known ground truth) we ask whether each signal ranks the vulnerable sink above the safe candidates. A prioritization oracle computes MRR, R@1, NDCG@ k , cost curves, a paired permutation test, and bootstrap confidence intervals. The result is falsifiable and reproducible: raw taint dominates (MRR 0.894, AUC 0.895), the fused structural field is slightly *worse* (0.872, $p = 5 \times 10^{-5}$, 95% CI [+0.0157, +0.0294]), and spectral (0.496) and topological (0.500) are at chance while curvature is below chance (0.376). A flat, near-acyclic servlet has no curvature or persistent topology to exploit, and taint is already near-optimal there.

3.2 Authorization, mechanized and operational

Let $F : V \rightarrow \mathbb{N}$ assign a privilege level; an edge $e = (u, v)$ is an *auth edge* iff $F(u) < F(v)$. A path with no auth edge satisfies $F(k) \leq F(s)$ — no privilege escalation — mechanized in Lean 4. The structural missing- authorization/BOLA detector flags a function with an HTTP source and a resource sink but no auth gate: on a curated 39-case corpus it reaches recall 1.0 / precision

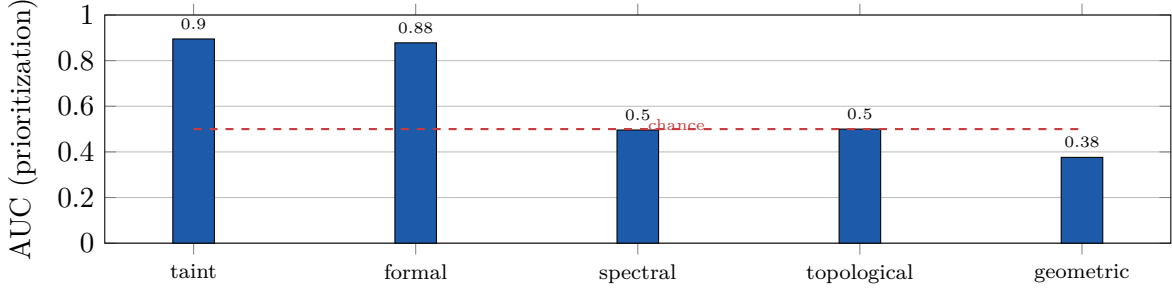


Figure 2: The boundary result: spectral and topological signals are at chance, curvature below it; taint is the dominant signal for injection prioritization.

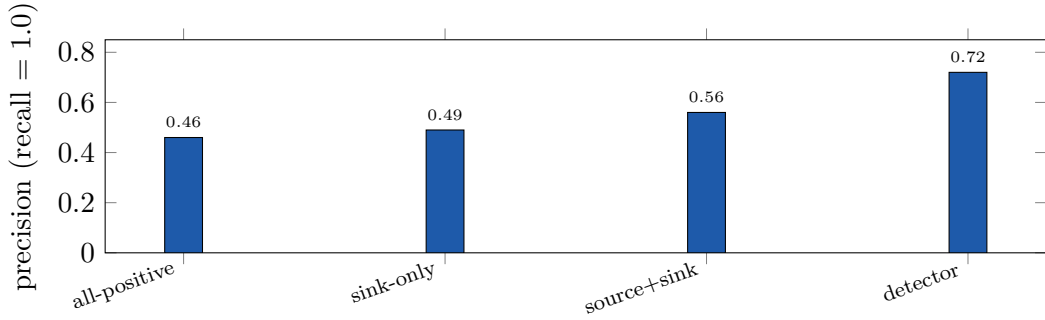


Figure 3: Each structural requirement of the BOLA/IDOR detector raises precision at constant recall: the detector reaches 0.72 where taint scores 0.

0.72 / F1 0.837 where taint scores 0, and a baseline ladder shows each requirement earns its keep (0.46 $\rightarrow$ 0.72). The Z3 verifier checks $\text{SAT}(\text{owner}(id) \neq \text{session})$ — an ownership witness that makes precise that `login_required` does not block BOLA, only an `owner` predicate does. On crAPI (44 operations, 9 documented BOLA/BFLA endpoints) an object-reference signal recovers all 9 (recall 1.0, precision 0.90) while the missing-auth check scores 0, because every BOLA endpoint *is* authenticated.

3.3 Directed path homology

Symmetrizing a directed graph is lossy in both directions: a 2-cycle $A \rightarrow B \rightarrow A$ collapses (a missed reentrancy) and a filled triangle becomes a false cycle. GLMY directed path homology resolves both: recall 1.0 / precision 0.75 on a 12-graph corpus versus 0.5/0.5 symmetrized, and on SmartBugs- curated smart contracts 0.968 vs. 0.065 recall. It returns the concrete β_1 generator (the cycle’s edges) as evidence; its two false positives are diamond path-space holes, not feedback loops — H_1 is a high-recall, necessary-but-not-sufficient superset of destructive cycles.

3.4 The neuro-symbolic arbitration boundary

On a 38-file corpus (29 confirmed findings) the offline heuristic and the LiteLLM-backed agent agree on detection and ranking on 37/38 files (agreement 0.974), while the LLM changes the CWE on 28% of findings and produces more concise titles. A spec-mining probe mines a plausible precondition every time (hallucination 0) at 0.5 token recall. The contribution is an architecture — LLM proposes, Z3 disposes — plus its measurement.

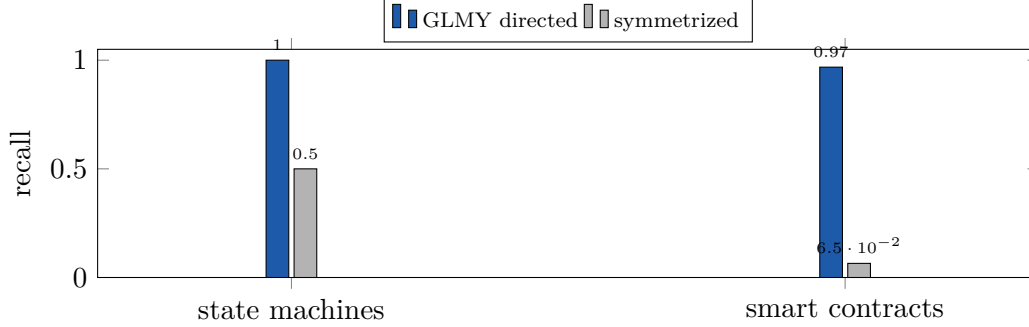


Figure 4: Directed (GLMY) path homology vs. symmetrized homology: symmetrization collapses the directed 2-cycle, dropping recall from 1.0 $\rightarrow$ 0.5 on state machines and 0.97 $\rightarrow$ 0.065 on smart contracts.

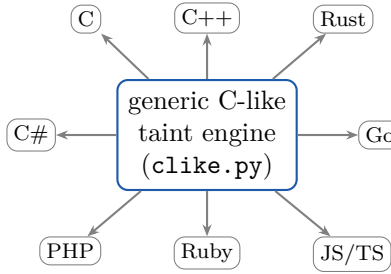


Figure 5: One generic taint engine drives nine tree-sitter grammars via per-language source/sink/sanitizer profiles; Python, Java (interprocedural) and Solidity have dedicated adapters.

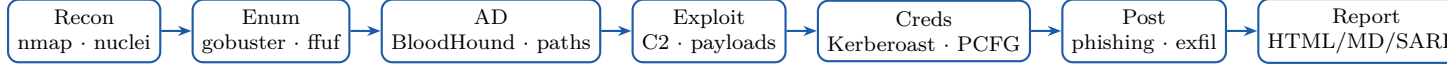
4 Multi-language SAST

One generic taint engine (`clike.py`) drives nine tree-sitter grammars (C, C++, Rust, C#, Go, PHP, Ruby, JavaScript/TypeScript), with per-language *profiles* that declare sources, sinks, sanitizers, and auth gates. The engine resolves callees (including receivers and scoped names), propagates taint through straight-line assignments, treats output-parameter sources (`fgets`, `gets`) and pointer declarators, and emits the canonical IR. A registry (`categories.py`) unifies severity, CWE, OWASP Top-10, default CVSS 3.1/4.0 vectors, MITRE tactic and remediation per category, so a finding is immediately enriched for reporting.

5 The red-team platform

VALEN is a single installable Python package whose server is a multi-user *team server*: users with roles (`viewer/operator/admin`), PBKDF2 sessions, per-route RBAC, engagements with membership, evidence artifacts, and Server-Sent Events for live operator collaboration, all persisted in SQLite. The red-team surface spans the kill chain:

- **Recon / enumeration** — nmap, masscan, gobuster, ffuf, nuclei wrappers with stealth profiles and CVE/KEV/EPSS intelligence.
- **Active Directory** — BloodHound/SharpHound ingestion into the IR, tier-0 attack paths, Kerberoast/AS-REP selection, GPP `cpassword` decryption, and impacket/kerbrute/hashcat command builders.



gated by scope allow-list, `-allow-exec`, RBAC, and operator approval

Figure 6: The red-team platform spans the kill chain end-to-end, coordinated by the team server and gated by an authorization-first model.

- **C2 and payloads** — Sliver integration (gRPC-first, shell fallback), msfvenom/handler/HTA planners, GoPhish and exfiltration planners.
- **Hybrid agent** — a deterministic kill-chain planner with an operator approval queue; intrusive actions require explicit `-authorize`.
- **Reporting** — HTML / Markdown / JSON / SARIF with CVSS 3.1/4.0 and CVE/KEV/EPSS enrichment.

Everything offensive is gated by a scope allow-list (loopback by default), an execution allow-list (`-allow-exec`), and per-operator roles — an *authorization-first* model, not an afterthought.

6 Attack-graph mathematics

The IR is a typed directed multigraph $G = (V, E)$ over principals. Four families of methods are layered over it.

6.1 Flow: chokepoints and disjoint plans

Menger min-cut. Given entry set S and target set T , we compute the minimum vertex set whose removal separates them, by s - t max flow on the vertex-split graph (each v becomes $v_{\text{in}} \rightarrow v_{\text{out}}$ of capacity 1; sources/targets unbounded) with Edmonds–Karp. The saturated split edges on the source side of the residual graph are the *obligatory chokepoints*. A flow $\geq \infty$ means a source is directly adjacent to a target (min-cut 0).

Min-cost flow. With a per-edge cost model (stealth + effort; MemberOf = 1, DCSync = 5), we send k units at minimum cost via successive shortest augmenting paths. Internal vertices have capacity 1, so the result is a set of *vertex-disjoint* cheapest routes — “with k compromises, the k cheapest independent ways in”.

Articulation points and bridges (Tarjan, on the symmetrized graph) rank the most disruptive principals and links — the attacker/disruption view — while the directed min-cut is the defender/hardening view.

6.2 Probability: hitting time

Model the graph as an absorbing Markov chain: targets are absorbing and each transient node transitions uniformly over out-neighbours. The hitting probability satisfies

$$h(u) = \mathbb{K}[u \in T] + \mathbb{K}[u \notin T] \cdot \frac{1}{\deg^+(u)} \sum_{v: u \rightarrow v} h(v),$$

computed by value iteration or *exactly* by Gaussian elimination on $(I - Q)h = r$. In the toy corpus an entry with three out-edges and one route to the target scores $\frac{1}{3}$, a direct entry 1 — a probabilistic risk ranking distinct from boolean reachability.

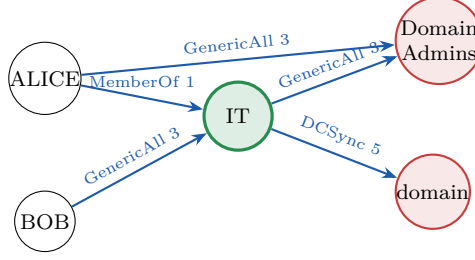


Figure 7: The AD attack graph: with entry BOB, IT is the Menger min-cut chokepoint (min-cut 1); ALICE reaches Domain Admins directly (min-cut 0). Least-cost plans: ALICE→Domain Admins (3.0) and ALICE→IT→domain (6.0); BOB is certified unreachable to Domain Admins.

6.3 Formal synthesis: minimal-cost exploit plans

Boolean reachability answers *is it reachable?*; we answer *what is the cheapest ordered sequence of actions?* Model the graph as a transition system

$$\text{has}(s+1, n) \equiv \text{has}(s, n) \vee \bigvee_{e: m \rightarrow n} (\text{taken}_e \wedge \text{has}(s, m)),$$

with entries compromised at step 0, and ask **z3.Optimize** to minimize $\sum_e \text{taken}_e \cdot \text{cost}(e)$ subject to reaching the target within k steps. Reconstructing the ordered sequence yields the technique-by-technique narrative. We extend it to **AND-preconditions**: compound actions {sources → target} that fire only when all sources are compromised (e.g. DCSync needs Domain Admins *and* a session on a Domain Controller).

6.4 Password guessing: PCFG

We implement Weir’s probabilistic context-free grammar: tokenize passwords into $L/D/S$ runs; the base structure (e.g. $L_6D_4S_1$) is the non-terminal, and terminals are keyed by (class, length) so no cross-length mixing. $P(\text{guess}) = P(\text{structure}) \cdot \prod P(\text{terminal})$, and guesses are emitted in strictly decreasing probability by a best-first walk. The training corpus is a cracked hashcat potfile.

7 Evaluation

7.1 Autonomous pentest against OWASP crAPI

Against a fresh crAPI 1.1.5 lab the autonomous agent solves **18/18** documented challenges — BOLA/BFLA, mass assignment, SSRF, NoSQL/SQL injection, JWT forgery, password reset, rate limit, and three LLM prompt-injection challenges (routed through a local LiteLLM proxy). The BOLA detector reaches recall 1.0 / precision 0.90 / F1 0.947.

7.2 Active Directory toy corpus

On a synthetic SharpHound corpus the layers compose (Figure 7): with entry BOB, min-cut 1 at the intermediate group IT; with entry ALICE (direct GenericAll on Domain Admins), min-cut 0; hitting ALICE = 1.0, BOB = $\frac{1}{3}$; synthesis yields ALICE $\xrightarrow{\text{GenericAll/T1098}}$ Domain Admins (cost 3.0) and ALICE $\xrightarrow{\text{MemberOf}}$ IT $\xrightarrow{\text{DCSync/T1003.006}}$ domain (cost 6.0), while BOB is certified *unreachable* to Domain Admins.

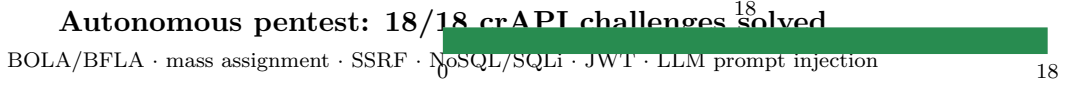


Figure 8: The autonomous agent solves all 18 documented crAPI challenges, including three LLM prompt-injection challenges routed through a local LiteLLM proxy.

7.3 Results at a glance

Contribution	Metric	Result
C1 boundary	taint vs. fused field (MRR)	0.894 vs. 0.872 ($p = 5 \times 10^{-5}$)
C2 authorization	crAPI BOLA/IDOR (recall / precision)	1.0 / 0.90
C2 authorization	curated BOLA ladder (precision)	0.46 $\rightarrow$ 0.72
C3 directed homology	GLMY vs. symmetrized (contracts)	0.968 vs. 0.065 recall
C4 arbitration	detection/ranking agreement	0.974; CWE 28%
C5 languages	SAST adapters	9 languages, 1 engine
C6 platform	team server	roles, RBAC, SSE, SQLite
C7 math	chokepoints / min-cost flow / hitting / synthesis / PCFG	implemented
Red team	autonomous crAPI challenges	18/18 solved

8 Limitations and ethics

The evaluations are a single real API plus curated corpora, not public benchmarks; the structural detector over-approximates; the Z3 witness holds under the *modeled* semantics; the pure-Python numeric fallback is an honest approximation of the Rust core; the PCFG prior is only as good as its corpus. The platform is an *authorized-engagement* tool: scope is loopback by default, execution is behind `-allow-exec`, intrusive actions require operator approval, and every module carries a “for authorized engagements only” notice.

9 Conclusion

VALEN joins a measured defensive thesis — structural invariants verify authorization, state, and trust properties, not injection prioritization — with an offensive platform whose differentiator is *quantitative attack-graph mathematics*: chokepoints, least-cost and disjoint plans, exact hitting probabilities, minimal-cost exploit synthesis with AND-preconditions, and a PCFG password prior. All code is installable, tested (383 tests), and documented, and all results are reproducible from the included artifacts.